

CLINIC MANAGEMENT SYSTEM

Database Design & Implementation Report

Academic Project Report — University Database Course

Mohamed Ahmed Aref	241001386
Sara Khalil Mohamed	241000394
Hajer Hossam Abdulbaki	241000988
Jana Hosam AbdelWahab	241000888
Adham Mohammed AIDakrany	241002505

Date: May 30, 2026

1. Project Overview

1.1 System Description

The Clinic Management System (CMS) is a full-stack web application designed to digitalise and streamline the administrative and clinical operations of a healthcare facility. The system provides a centralised platform through which patients, doctors, clinics, departments, and appointments are managed in a structured, efficient manner. It replaces manual paper-based processes with an interactive dashboard accessible through a web browser.

1.2 Main Objectives

The primary objectives of the Clinic Management System are as follows:

- Centralise all patient demographic information in a single relational database, enabling fast retrieval and update of records.
- Automate appointment scheduling while preventing double-booking conflicts for doctors on the same date and time.
- Provide analytical insights, such as diagnosis trend tracking and total revenue calculation, to support clinical decision-making.
- Establish clear organisational hierarchy by linking doctors and clinics to medical departments.
- Offer a modern, responsive web interface with real-time data operations (create, read, delete) for all major entities.

1.3 Key Functionalities

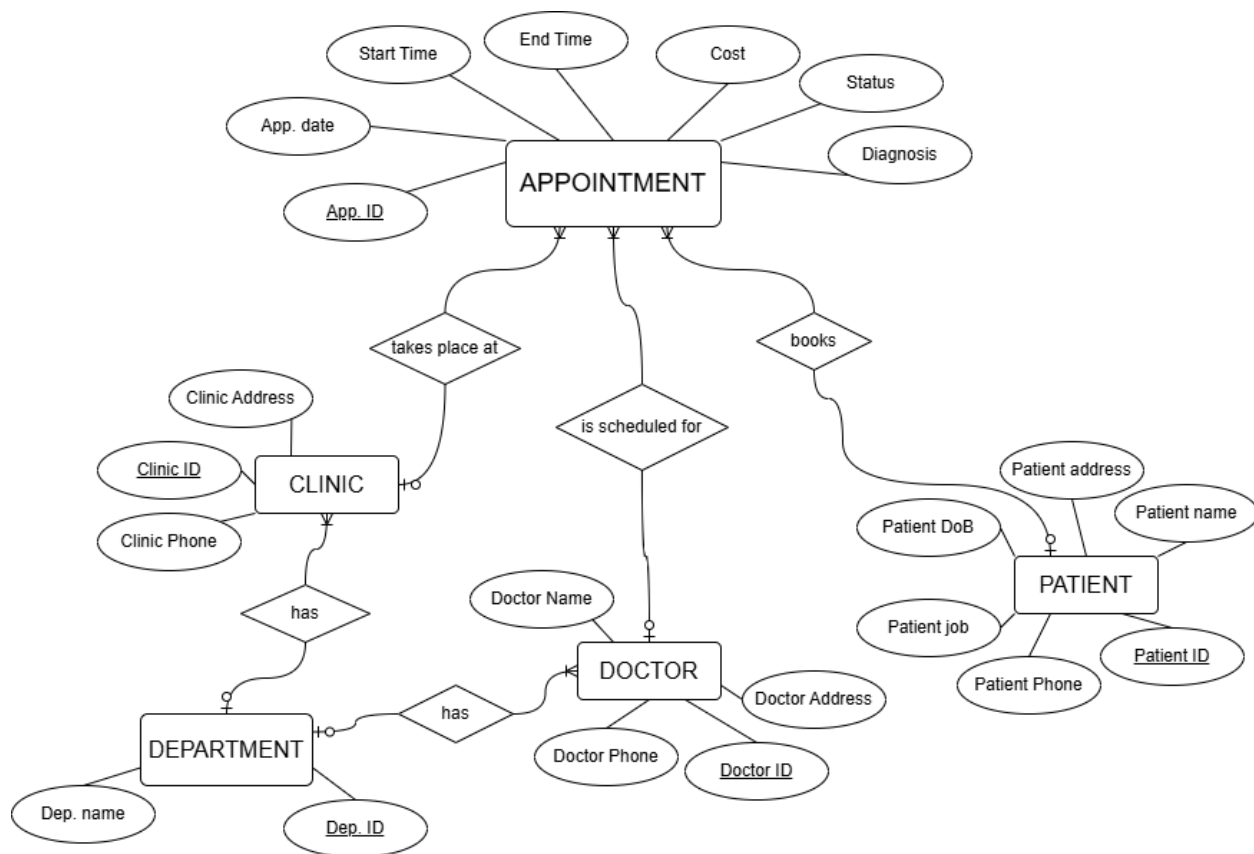
Feature	Description
Patient Management	Add, view, and delete patient records including contact details, birth date, and occupation.
Doctor Management	Register doctors linked to a specific medical department; view and delete records.
Clinic Management	Add and manage clinic locations, each associated with a department.
Department Management	Maintain a catalogue of 20 medical specialities; link to clinics.
Appointment Scheduling	Create appointments with overlap prevention; track status (scheduled, in progress, postponed, completed).
Analytics Dashboard	Real-time counters for all entities; diagnosis frequency trends; total revenue aggregation.
Overlap Prevention	Backend logic rejects new appointments that conflict with an existing time slot for the same doctor.

1.4 Technology Stack

Layer	Technology	Purpose
-------	------------	---------

Database	MySQL 8.0	Relational storage; DDL/DML; InnoDB engine for FK enforcement
Backend	Node.js + Express.js	RESTful API server; business logic; DB queries via mysql2 driver
Frontend	React.js	Single-page application; component-based UI; fetch API for HTTP calls
Styling	Custom CSS (Flexbox/Grid)	Dark-theme dashboard with animated transitions

2. Conceptual Design



The conceptual design phase identifies all real-world entities that must be represented in the database. Each entity encapsulates a distinct concept relevant to the clinic domain. The following subsections describe each entity with its attributes, primary key, and governing business rules.

2.1 Entity: PATIENT

Attribute	Data Type	Constraint	Description
-----------	-----------	------------	-------------

patient_id	INT	PRIMARY KEY	Unique numeric identifier for each patient
name	VARCHAR(100)	NOT NULL	Full legal name of the patient
phone	VARCHAR(20)	NULL allowed	Contact phone number
address	VARCHAR(200)	NULL allowed	Residential address
birth_date	DATE	NULL allowed	Date of birth for age computation
job	VARCHAR(100)	NULL allowed	Occupation/profession of the patient

Business Rules:

- A patient may have zero or many appointments over time.
- patient_id is system-assigned and does not change after creation.

2.2 Entity: DOCTOR

Attribute	Data Type	Constraint	Description
doctor_id	INT	PRIMARY KEY	Unique numeric identifier for each doctor
name	VARCHAR(100)	NOT NULL	Full name including title (e.g., Dr.)
phone	VARCHAR(20)	NULL allowed	Contact phone number
address	VARCHAR(200)	NULL allowed	Work or home address
dept_id	INT	FOREIGN KEY → DEPARTMENT	Specialisation department of the doctor

Business Rules:

- Every doctor must belong to exactly one department (dept_id may be NULL in the current schema, indicating an unassigned doctor).
- A doctor may conduct multiple appointments across different dates.

2.3 Entity: DEPARTMENT

Attribute	Data Type	Constraint	Description
dept_id	INT	PRIMARY KEY	Unique numeric identifier for each department
name	VARCHAR(100)	NOT NULL	Name of the medical speciality (e.g., Cardiology)

Business Rules:

- Each department may house multiple doctors and multiple clinics.
- The system currently holds 20 distinct departments covering major medical specialities.

2.4 Entity: CLINIC

Attribute	Data Type	Constraint	Description
clinic_id	INT	PRIMARY KEY	Unique numeric identifier for each clinic
name	VARCHAR(100)	NOT NULL	Commercial or official name of the clinic
address	VARCHAR(200)	NULL allowed	Physical location/address of the clinic
dept_id	INT	FOREIGN KEY → DEPARTMENT	Department to which this clinic belongs

Business Rules:

- A clinic belongs to exactly one department.
- A clinic may host multiple appointments.

2.5 Entity: APPOINTMENT

Attribute	Data Type	Constraint	Description
appointment_id	INT	PRIMARY KEY	Unique identifier for the appointment
appointment_date	DATE	NOT NULL	Calendar date of the appointment
start_time	TIME	NULL allowed	Scheduled start time
end_time	TIME	NULL allowed	Scheduled end time
cost	DECIMAL(10,2)	NULL allowed	Fee charged for the appointment
status	ENUM	NULL allowed	Current state: scheduled / in progress / postponed / completed
diagnosis	TEXT	NULL allowed	Free-text clinical diagnosis or reason for visit
patient_id	INT	FOREIGN KEY → PATIENT	The patient attending the appointment
doctor_id	INT	FOREIGN KEY → DOCTOR	The doctor conducting the appointment
clinic_id	INT	FOREIGN KEY → CLINIC	The clinic where the appointment takes place

Business Rules:

- An appointment must be associated with at least a patient and a doctor.

- The status lifecycle is: scheduled → in progress → completed (or postponed at any stage).
- No two appointments for the same doctor should overlap in time on the same date.

3. Entity-Relationship (ER) Design

3.1 Overview of the ER Model

The ER model for the Clinic Management System comprises five strong entities: PATIENT, DOCTOR, DEPARTMENT, CLINIC, and APPOINTMENT. There are no weak entities in this design because every entity has a natural or surrogate primary key. APPOINTMENT functions as a central associative entity that connects PATIENT, DOCTOR, and CLINIC.

3.2 Relationships

Relationship	Entity A	Entity B	Cardinality	Participation A	Participation B	Explanation
BELONGS_TO	DOCTOR	DEPARTMENT	N:1	Partial	Total	Many doctors can belong to one department ; every doctor should have a speciality.
BELONGS_TO	CLINIC	DEPARTMENT	N:1	Partial	Total	Many clinics can belong to one department (e.g., two cardiology clinics).
BOOKED_FOR	APPOINTMENT	PATIENT	N:1	Total	Partial	Each appointment is booked for exactly one patient; a patient may have multiple appointments.
CONDUCTED_BY	APPOINTMENT	DOCTOR	N:1	Total	Partial	Each appointment is conducted by one doctor; a doctor conducts many

						appointments.
HELD_AT	APPOINTMENT	CLINIC	N:1	Partial	Partial	An appointment may be held at a specific clinic; a clinic hosts many appointments.

3.3 Entity Classification

Entity	Type	Reason
PATIENT	Strong	Has a natural surrogate key (patient_id); exists independently.
DOCTOR	Strong	Has its own primary key (doctor_id); exists independently.
DEPARTMENT	Strong	Has its own primary key (dept_id); does not depend on any other entity.
CLINIC	Strong	Has its own primary key (clinic_id); references DEPARTMENT but is not weak.
APPOINTMENT	Strong / Associative	Has its own PK (appointment_id) but links PATIENT, DOCTOR, and CLINIC — functions as a central junction entity.

3.4 Textual ERD Description

The following description is sufficient to recreate the ER diagram in any modelling tool:

```

DEPARTMENT(dept_id [PK], name)
DOCTOR(doctor_id [PK], name, phone, address, dept_id [FK → DEPARTMENT])
CLINIC(clinic_id [PK], name, address, dept_id [FK → DEPARTMENT])
PATIENT(patient_id [PK], name, phone, address, birth_date, job)
APPOINTMENT(appointment_id [PK], appointment_date, start_time, end_time, cost, status, diagnosis,
              patient_id [FK → PATIENT], doctor_id [FK → DOCTOR],
              clinic_id [FK → CLINIC])

```

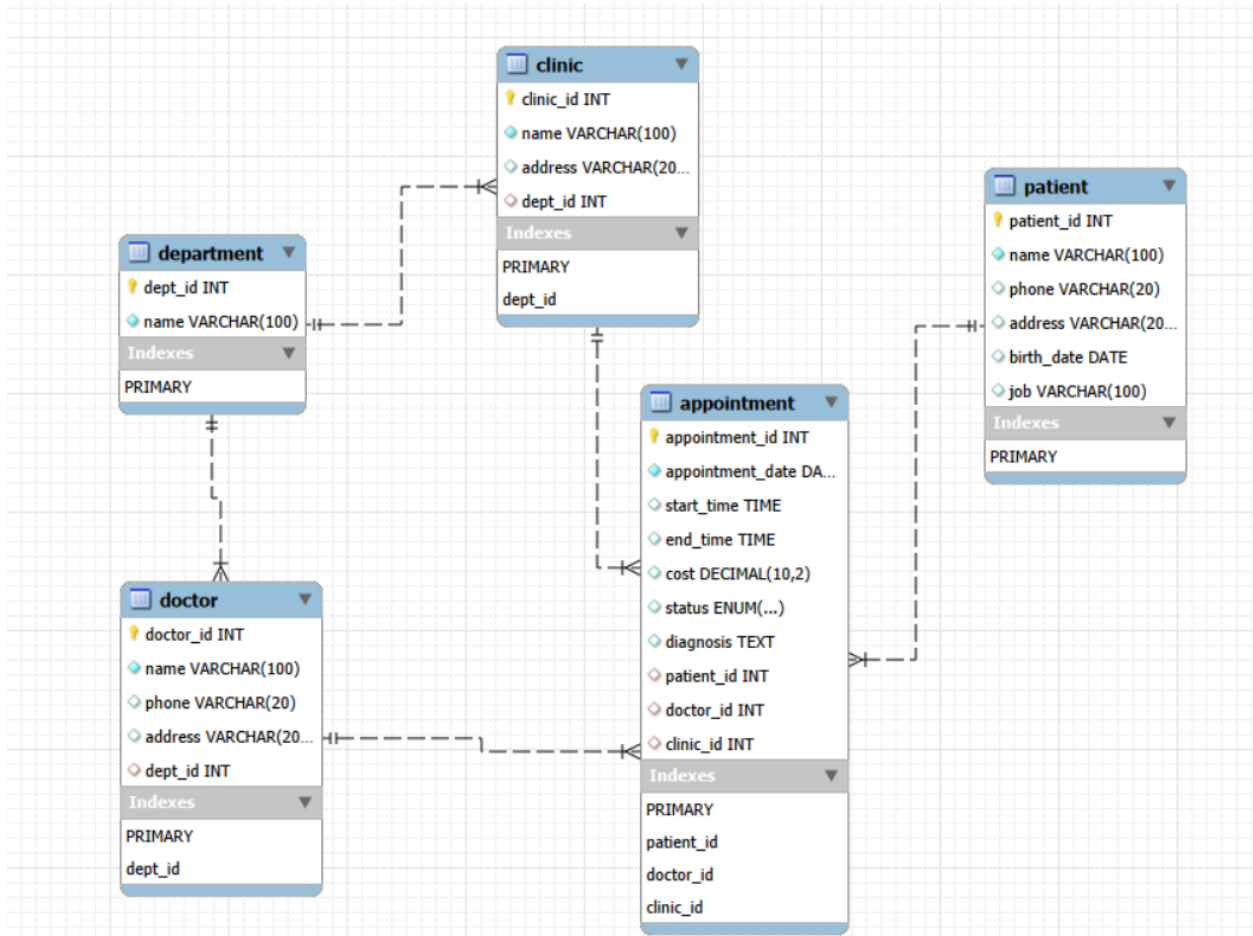
Relationships:

- DOCTOR — N:1 —→ DEPARTMENT (via dept_id)
- CLINIC — N:1 —→ DEPARTMENT (via dept_id)

- APPOINTMENT — N:1 —→ PATIENT (via patient_id)
- APPOINTMENT — N:1 —→ DOCTOR (via doctor_id)
- APPOINTMENT — N:1 —→ CLINIC (via clinic_id)

4. Relational Schema Design

Converting the ER model to relational schemas yields the following tables. Foreign keys are shown with their referenced table in parentheses.



4.1 Schema Notation

```
DEPARTMENT (  
    dept_id      INT          PRIMARY KEY,  
    name         VARCHAR(100) NOT NULL  
)  
  
PATIENT(  
    patient_id   INT          PRIMARY KEY,  
    name         VARCHAR(100) NOT NULL,  
    phone        VARCHAR(20),  
    address      VARCHAR(200),  
    birth_date   DATE,  
    job          VARCHAR(100)  
)
```

```

DOCTOR(
    doctor_id    INT            PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    phone        VARCHAR(20),
    address      VARCHAR(200),
    dept_id      INT            REFERENCES DEPARTMENT(dept_id)
)

CLINIC(
    clinic_id    INT            PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    address      VARCHAR(200),
    dept_id      INT            REFERENCES DEPARTMENT(dept_id)
)

APPOINTMENT(
    appointment_id INT            PRIMARY KEY,
    appointment_date DATE        NOT NULL,
    start_time     TIME,
    end_time       TIME,
    cost           DECIMAL(10,2),
    status         ENUM('scheduled', 'in progress', 'postponed', 'completed'),
    diagnosis      TEXT,
    patient_id     INT            REFERENCES PATIENT(patient_id),
    doctor_id      INT            REFERENCES DOCTOR(doctor_id),
    clinic_id      INT            REFERENCES CLINIC(clinic_id)
)

```

4.2 Schema Summary Table

Table	PK	Foreign Keys	Notable Constraints
DEPARTMENT	dept_id	—	name NOT NULL
PATIENT	patient_id	—	name NOT NULL
DOCTOR	doctor_id	dept_id → DEPARTMENT	name NOT NULL
CLINIC	clinic_id	dept_id → DEPARTMENT	name NOT NULL
APPOINTMENT	appointment_id	patient_id → PATIENT, doctor_id → DOCTOR, clinic_id → CLINIC	appointment_date NOT NULL; status ENUM

5. Database Constraints

5.1 Entity Integrity Constraints

Entity integrity requires that every table has a primary key and that the primary key column(s) never contain NULL values. In this schema:

- DEPARTMENT.dept_id, PATIENT.patient_id, DOCTOR.doctor_id, CLINIC.clinic_id, and APPOINTMENT.appointment_id are all declared as INT NOT NULL PRIMARY KEY.
- MySQL automatically enforces uniqueness and non-nullability on primary key columns.

5.2 Referential Integrity Constraints

Referential integrity is enforced through FOREIGN KEY ... REFERENCES declarations, backed by InnoDB storage engine:

Foreign Key	Child Table	Parent Table	Effect of Violation
doctor.dept_id	DOCTOR	DEPARTMENT	INSERT/UPDATE rejected if dept_id does not exist in DEPARTMENT
clinic.dept_id	CLINIC	DEPARTMENT	INSERT/UPDATE rejected if dept_id does not exist in DEPARTMENT
appointment.patient_id	APPOINTMENT	PATIENT	INSERT/UPDATE rejected if patient_id does not exist in PATIENT
appointment.doctor_id	APPOINTMENT	DOCTOR	INSERT/UPDATE rejected if doctor_id does not exist in DOCTOR
appointment.clinic_id	APPOINTMENT	CLINIC	INSERT/UPDATE rejected if clinic_id does not exist in CLINIC

The dump file temporarily disables foreign key checks (FOREIGN_KEY_CHECKS=0) during bulk insert operations to allow tables to be loaded in any order. Re-enabling the checks after loading ensures all references remain valid.

5.3 Domain Constraints

- appointment_date is declared DATE NOT NULL, ensuring every appointment has a valid calendar date.
- status is an ENUM limited to four values: 'scheduled', 'in progress', 'postponed', 'completed'. Any other value is rejected by MySQL automatically.
- cost is DECIMAL(10,2), ensuring monetary values are stored with exactly two decimal places.
- phone columns are VARCHAR(20), accommodating international formats.

5.4 Business Constraints

- Appointment Overlap Prevention: The backend API implements a time-overlap check before inserting a new appointment. The query verifies that no existing appointment for the same

doctor on the same date overlaps the proposed start/end window. This is a business constraint enforced at the application layer.

- Status Lifecycle: The ENUM constraint implicitly enforces the valid status values, preventing invalid state transitions from being stored.
- Name Non-Nullability: The name column in PATIENT, DOCTOR, DEPARTMENT, and CLINIC is NOT NULL, ensuring that no anonymous records can be created.

6. SQL Implementation — DDL

6.1 Database Creation

The dump targets the database clinicdb (ClinicDB) using charset utf8mb4 with collation utf8mb4_0900_ai_ci, which supports the full Unicode character set including emoji and multilingual characters — appropriate for a clinic system serving diverse patient populations.

6.2 CREATE TABLE: department

```
CREATE TABLE `department` (  
  `dept_id` int NOT NULL,  
  `name` varchar(100) NOT NULL,  
  PRIMARY KEY (`dept_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Explanation:

dept_id is a simple integer acting as the surrogate primary key. name is forced NOT NULL because a department without a name would be meaningless in a clinical context. InnoDB engine is chosen for full foreign key support.

6.3 CREATE TABLE: patient

```
CREATE TABLE `patient` (  
  `patient_id` int NOT NULL,  
  `name` varchar(100) NOT NULL,  
  `phone` varchar(20) DEFAULT NULL,  
  `address` varchar(200) DEFAULT NULL,  
  `birth_date` date DEFAULT NULL,  
  `job` varchar(100) DEFAULT NULL,  
  PRIMARY KEY (`patient_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Explanation:

patient_id is the PK. Only name is mandatory; all other attributes are optional (DEFAULT NULL) to accommodate scenarios where information is not yet known at registration. birth_date uses the DATE type for accurate age computation. job is stored as a free-text string, avoiding a separate lookup table and keeping the schema simple at this stage.

6.4 CREATE TABLE: doctor

```
CREATE TABLE `doctor` (  
  `doctor_id` int NOT NULL,  
  `name` varchar(100) NOT NULL,  
  `phone` varchar(20) DEFAULT NULL,  
  `address` varchar(200) DEFAULT NULL,  
  `dept_id` int DEFAULT NULL,  
  PRIMARY KEY (`doctor_id`),  
  KEY `dept_id` (`dept_id`),
```

```

    CONSTRAINT `doctor_ibfk_1` FOREIGN KEY (`dept_id`) REFERENCES `department`
    (`dept_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Explanation:

doctor_id is the PK. dept_id is a nullable FK linking the doctor to a department, using InnoDB's FOREIGN KEY constraint. A secondary index (KEY) is automatically created on dept_id to optimise JOIN queries between doctor and department. The ON DELETE / ON UPDATE action defaults to RESTRICT, preventing orphaned doctor records.

6.5 CREATE TABLE: clinic

```

CREATE TABLE `clinic` (
  `clinic_id` int NOT NULL,
  `name`      varchar(100) NOT NULL,
  `address`   varchar(200) DEFAULT NULL,
  `dept_id`   int           DEFAULT NULL,
  PRIMARY KEY (`clinic_id`),
  KEY `dept_id` (`dept_id`),
  CONSTRAINT `clinic_ibfk_1` FOREIGN KEY (`dept_id`) REFERENCES `department`
  (`dept_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Explanation:

Structurally mirrors the doctor table in its use of a nullable dept_id FK. This allows a clinic to exist temporarily without departmental assignment. The secondary index on dept_id accelerates queries joining clinic to department.

6.6 CREATE TABLE: appointment

```

CREATE TABLE `appointment` (
  `appointment_id` int NOT NULL,
  `appointment_date` date NOT NULL,
  `start_time`     time DEFAULT NULL,
  `end_time`       time DEFAULT NULL,
  `cost`           decimal(10,2) DEFAULT NULL,
  `status`         enum('scheduled','in progress','postponed','completed') DEFAULT
NULL,
  `diagnosis`      text,
  `patient_id`     int DEFAULT NULL,
  `doctor_id`      int DEFAULT NULL,
  `clinic_id`      int DEFAULT NULL,
  PRIMARY KEY (`appointment_id`),
  KEY `patient_id` (`patient_id`),
  KEY `doctor_id`  (`doctor_id`),
  KEY `clinic_id`  (`clinic_id`),
  CONSTRAINT `appointment_ibfk_1` FOREIGN KEY (`patient_id`) REFERENCES `patient`
  (`patient_id`),
  CONSTRAINT `appointment_ibfk_2` FOREIGN KEY (`doctor_id`) REFERENCES `doctor`
  (`doctor_id`),

```

```
CONSTRAINT `appointment_ibfk_3` FOREIGN KEY (`clinic_id`) REFERENCES `clinic`  
(`clinic_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Explanation:

This is the most complex table in the schema. `appointment_id` is the surrogate PK. `appointment_date` is NOT NULL, making date mandatory for scheduling. Three FK columns (`patient_id`, `doctor_id`, `clinic_id`) reference their respective parent tables. Three secondary indexes are created automatically on each FK column to support JOIN and WHERE clause performance. The ENUM type for status limits entries to four valid lifecycle values.

7. SQL Implementation — DML

7.1 INSERT Statements

The dump file contains bulk INSERT statements for all five tables. Below are representative examples with explanations.

7.1.1 Inserting Departments

```
INSERT INTO `department` VALUES
  (1, 'Cardiology'),
  (2, 'Gastroenterology'),
  (3, 'Dermatology'),
  ... (20 rows total);
```

This populates the lookup table with 20 predefined medical specialities. dept_id values run from 1 to 20. No FK dependencies exist at this level, so department data is inserted first.

7.1.2 Inserting Patients

```
INSERT INTO `patient` VALUES
  (12501, 'Patient_1', '01020194829', 'Giza', '1999-04-19', 'Designer'),
  (12502, 'Patient_2', '01067107704', 'Giza', '1986-04-19', 'Designer'),
  ... (20 rows total);
```

Patient IDs begin at 12501, creating a realistic ID range. birth_date uses ISO format (YYYY-MM-DD). All patients share the same address 'Giza' in the test dataset, indicating the data was auto-generated.

7.1.3 Inserting Appointments

```
INSERT INTO `appointment` VALUES
  (1, '2025-09-06', '09:00:00', '10:30:00', 458.59, 'scheduled', 'heart
disease', 12516, 1020, NULL),
  (2, '2024-09-18', '15:00:00', '09:30:00', 528.21, 'completed', 'fatty
liver', 12504, 1010, NULL),
  ...;
```

Each appointment row specifies all nine non-PK columns. clinic_id is NULL for all sample rows, indicating no clinic has been linked to these test appointments. cost is a decimal value representing the appointment fee. diagnosis contains free-text clinical terms.

7.2 UPDATE Statements

No explicit UPDATE statements appear in the provided dump. However, the application supports updates implicitly. In a production extension, the following pattern would be used:

```
-- Update appointment status when doctor begins consultation
UPDATE appointment
SET    status = 'in progress'
WHERE  appointment_id = 1;

-- Update patient phone number
```

```
UPDATE patient
SET    phone = '01099999999'
WHERE  patient_id = 12501;
```

7.3 DELETE Statements

The backend API implements DELETE endpoints for patients, doctors, clinics, and departments. The SQL executed is:

```
DELETE FROM Patient    WHERE patient_id    = ?;
DELETE FROM Doctor     WHERE doctor_id     = ?;
DELETE FROM Clinic     WHERE clinic_id     = ?;
DELETE FROM Department WHERE department_id = ?;
```

Because InnoDB enforces referential integrity, deleting a patient who has existing appointments will fail unless those appointments are deleted first (RESTRICT default). This protects data consistency.

8. Test Data

8.1 DEPARTMENT — Sample Records

dept_id	name
1	Cardiology
2	Gastroenterology
3	Dermatology
4	Neurology
5	Orthopedics
6	Pediatrics
7	Oncology
8	ENT
9	Ophthalmology
10	Urology
11	Psychiatry
12	Endocrinology

Rationale: 20 real-world departments are seeded to test FK lookups from both DOCTOR and CLINIC. They span major clinical specialties, ensuring diverse JOIN results.

8.2 PATIENT — Sample Records

patient_id	name	phone	address	birth_date	job
12501	Patient_1	01020194829	Giza	1999-04-19	Designer
12502	Patient_2	01067107704	Giza	1986-04-19	Designer
12503	Patient_3	01073356192	Giza	1973-04-19	Engineer
12504	Patient_4	01026587589	Giza	1977-04-19	Doctor
12505	Patient_5	01076390074	Giza	1992-04-19	Designer
12506	Patient_6	01013341232	Giza	1984-04-19	Designer
12507	Patient_7	01014766721	Giza	1996-04-19	Doctor
12508	Patient_8	01031124553	Giza	1974-04-19	Doctor
12509	Patient_9	01041263008	Giza	1969-04-19	Designer
12510	Patient_10	01050677858	Giza	1984-04-19	Teacher

Rationale: Ten records spanning birth years 1969–1999, covering a range of working-age adults. Job diversity (Designer, Engineer, Doctor, Teacher, Student) validates that the job field stores free-text correctly.

8.3 DOCTOR — Sample Records

doctor_id	name	phone	address	dept_id
1001	Dr. Doctor_1	01052277311	Cairo	18
1002	Dr. Doctor_2	01094776010	Cairo	2
1003	Dr. Doctor_3	01070248431	Cairo	1
1004	Dr. Doctor_4	01031736135	Cairo	2
1005	Dr. Doctor_5	01060750713	Cairo	13
1006	Dr. Doctor_6	01058638375	Cairo	16
1007	Dr. Doctor_7	01027372738	Cairo	14
1008	Dr. Doctor_8	01082336668	Cairo	20
1009	Dr. Doctor_9	01058042095	Cairo	15
1010	Dr. Doctor_10	01091769824	Cairo	9

Rationale: Doctors reference various dept_id values, validating the FK constraint to DEPARTMENT. Multiple doctors share the same department (e.g., doctors 2 and 4 both belong to dept 2 — Gastroenterology), demonstrating the N:1 cardinality.

8.4 CLINIC — Sample Records

clinic_id	name	address	dept_id
101	Heart Care Clinic	Cairo - Nasr City	1
102	Cardio Plus	Giza - Dokki	1
103	Digestive Clinic	Cairo - Maadi	2
104	Skin Care	Cairo - Heliopolis	3
105	Brain Clinic	Giza - Mohandessin	4
106	Bone Clinic	Cairo - 6th October	5
107	Kids Care	Giza - Haram	6
108	Cancer Center	Cairo - Abbasiya	7
109	ENT Care	Cairo - Zamalek	8
110	Eye Center	Giza - Sheikh Zayed	9

Rationale: Two clinics share dept_id=1 (Cardiology), confirming the 1:N relationship between DEPARTMENT and CLINIC. Cairo and Giza locations represent the real Egyptian metropolitan area served by the system.

8.5 APPOINTMENT — Sample Records

id	date	start	end	cost	status	diagnosis	patient_id	doctor_id
1	2025-09-06	09:00	10:30	458.59	scheduled	heart disease	12516	1020
2	2024-09-18	15:00	09:30	528.21	completed	fatty liver	12504	1010
3	2024-03-07	11:00	16:30	530.19	scheduled	heart disease	12503	1008
5	2024-03-18	08:00	16:30	558.08	in progress	fatty liver	12505	1013
6	2024-12-08	12:00	13:30	235.32	in progress	diabetes	12502	1011
7	2025-03-27	10:00	13:30	722.19	postponed	diabetes	12515	1013
10	2023-09-26	12:00	15:30	586.55	completed	migraine	12508	1010
11	2025-04-25	10:00	12:30	364.27	completed	diabetes	12515	1020
15	2025-01-04	13:00	09:30	364.54	completed	heart disease	12515	1015
20	2025-06-15	09:00	16:30	941.94	completed	fatty liver	12518	1017

Rationale: All four status values appear in the sample, validating ENUM storage. Multiple diagnoses repeat (diabetes appears in 6 records), enabling meaningful GROUP BY aggregation tests. Costs range from 235.32 to 941.94, providing realistic revenue totals for analytics testing.

9. Backend Analysis

9.1 Architecture Overview

The backend is a Node.js application built with the Express.js framework. It follows a simple, single-file MVC-lite structure where route definitions, controller logic, and database queries are co-located. The `mysql2` package is used for MySQL connectivity, providing a callback-based query interface. CORS middleware is enabled globally, allowing the React frontend to issue cross-origin requests from any origin. The server listens on port 3001.

9.2 Database Connection

```
const db = mysql.createConnection({
  host: 'localhost',  user: 'root',
  password: '1234',   database: 'ClinicDB'
});
db.connect(() => console.log('DB Connected'));
```

A single persistent connection is created at startup using `createConnection`. In a production environment, a connection pool (`mysql.createPool`) would be preferable to handle concurrent requests and connection failures gracefully. The current approach is suitable for a single-user academic demonstration.

9.3 API Endpoints

Method	Endpoint	Description	Request Body	Response
GET	/	Health check	—	{ message: 'Clinic API Running' }
GET	/patients	Retrieve all patients	—	JSON array of patient records, ordered by patient_id DESC
POST	/patients	Add a new patient	{ name, phone, address, birth_date, job }	{ message: 'Patient added' }
DELETE	/patients/:id	Remove a patient by ID	—	{ message: 'Patient deleted' }
GET	/doctors	Retrieve all doctors	—	JSON array of doctor records
POST	/doctors	Add a new doctor	{ name, specialization, phone }	{ message: 'Doctor added' }
DELETE	/doctors/:id	Remove a doctor by ID	—	{ message: 'Doctor deleted' }
GET	/clinics	Retrieve all clinics	—	JSON array of clinic records

POST	/clinics	Add a new clinic	{ name, location }	{ message: 'Clinic added' }
DELETE	/clinics/:id	Remove a clinic by ID	—	{ message: 'Clinic deleted' }
GET	/departments	All departments with clinic name	—	JSON with joined clinic_name
POST	/departments	Add a department	{ name, clinic_id }	{ message: 'Department added' }
DELETE	/departments/:id	Remove a department by ID	—	{ message: 'Department deleted' }
GET	/appointments	All appointments with patient & doctor names	—	JSON with joined patient_name, doctor_name
GET	/analytics/diagnosis-trend	Diagnosis frequency count	—	Array of { diagnosis, count }
GET	/analytics/revenue	Total revenue from appointments	—	{ total_revenue: X }

9.4 Appointment Overlap Prevention

```
const checkQuery = `
  SELECT * FROM Appointment
  WHERE doctor_id = ?
  AND appointment_date = ?
  AND (
    (start_time < ? AND end_time > ?) -- proposed slot starts inside existing
    OR
    (start_time < ? AND end_time > ?) -- existing slot starts inside proposed
  )`;
db.query(checkQuery, [doctor_id, appointment_date, end_time, start_time, end_time, start_time],
  (err, result) => {
    if (result.length > 0) return res.status(400).send('Time slot not available');
    // proceed with INSERT
  });
```

This overlap check implements the standard interval intersection test. Two intervals [A_start, A_end] and [B_start, B_end] overlap if A_start < B_end AND B_start < A_end. If any conflicting appointment is found (result.length > 0), the API returns HTTP 400 and rejects the booking. This logic executes before the INSERT, effectively acting as an application-level constraint.

9.5 Error Handling

Each database callback checks the err parameter. If an error occurs, the handler returns HTTP 500 with the error object serialised to JSON. The appointments GET endpoint additionally

catches errors silently and returns an empty array, preventing the dashboard from breaking when data is unavailable. Input validation is minimal in the current implementation — production code would add length checks, phone format validation, and date range validation.

10. Frontend Analysis

10.1 Application Structure

The frontend is a React single-page application contained within a single component file (App.js) alongside a global stylesheet (styles.css). The application is structured around a tab-based navigation system, where each tab renders a different section of the clinic management interface.

10.2 State Management

State Variable	Type	Purpose
tab	String	Controls which section is rendered (dashboard, patients, doctors, clinics, departments, appointments)
patients	Array	Stores all patient records fetched from GET /patients
doctors	Array	Stores all doctor records fetched from GET /doctors
clinics	Array	Stores all clinic records fetched from GET /clinics
departments	Array	Stores all department records fetched from GET /departments
appointments	Array	Stores all appointment records fetched from GET /appointments
pName/pPhone/pAddress/pBirth/pJob	String	Controlled input state for the Add Patient form
dName/dSpec/dPhone	String	Controlled input state for the Add Doctor form
cName/cLocation	String	Controlled input state for the Add Clinic form
deptName/deptClinic	String	Controlled input state for the Add Department form

10.3 Data Fetching Pattern

```
const loadAll = () => {
  fetch(API + '/patients').then(r => r.json()).then(d =>
    setPatients(Array.isArray(d) ? d : []));
  fetch(API + '/doctors') .then(r => r.json()).then(d => setDoctors
    (Array.isArray(d) ? d : []));
  fetch(API + '/clinics') .then(r => r.json()).then(d => setClinics
    (Array.isArray(d) ? d : []));
  // ... departments, appointments
};
useEffect(() => loadAll(), []);
```

The loadAll function fires five parallel fetch calls on component mount (useEffect with empty dependency array). The Array.isArray guard prevents crashes when an API returns an error object instead of an array. loadAll is also called after every mutation (add/delete) to refresh the displayed data.

10.4 Dashboard Tab

The dashboard displays five metric cards showing the count of patients, doctors, clinics, departments, and appointments. Counts are derived directly from the length of the corresponding state arrays, providing a live summary without additional API calls.

10.5 CRUD Interactions

Add operations: Each entity tab contains a form with controlled inputs. On clicking the Add button, the corresponding async function (e.g., `addPatient`) sends a POST request with JSON body, then calls `loadAll` to refresh the table.

Delete operations: Each table row has a Delete button that fires a DELETE request to the appropriate endpoint (e.g., `/patients/12501`) and refreshes the data. No confirmation dialog is present — this is a notable UI improvement opportunity.

10.6 Appointments Tab

The Appointments tab is read-only — it displays all appointments in a table with columns for ID, Patient, Doctor, Date, and Time. Patient and doctor names are resolved via the backend JOIN query (`patient_name` and `doctor_name` fields). The frontend uses the fallback expression `a.patient_name || a.patient` to handle both naming conventions from different API versions.

10.7 Styling

The CSS defines a dark-themed layout using a deep navy background (`#0b1220`) with white text. Key design decisions include:

- Flexbox layout (`.layout`) with a fixed 240px sidebar and a `flex:1` main panel.
- CSS grid (`.cards`) with auto-fit columns for responsive dashboard cards.
- Transition animations on sidebar items (`translateX` on hover) and cards (`translateY` on hover).
- A `fadeIn @keyframes` animation applied to `.main` for smooth tab switching.
- Custom scrollbar styling via `::-webkit-scrollbar` for aesthetic consistency.

11. Design Decisions & Rationale

11.1 Table Design

Each of the five tables was created to represent a distinct, non-overlapping concept. DEPARTMENT and CLINIC are separated (rather than merging them) because a department is a logical grouping of specialities, whereas a clinic is a physical location. This separation satisfies 3NF and avoids update anomalies — renaming a department does not require updating every clinic row.

The DOCTOR table stores dept_id as a simple FK rather than a many-to-many table. This reflects the real-world constraint that a doctor has one primary speciality. If multi-speciality doctors need to be modelled in future, a junction table DOCTOR_DEPARTMENT would be introduced.

11.2 Primary Key Selection

All primary keys are integer surrogate keys. This decision was made for performance (integer comparisons are faster than string comparisons on JOIN operations), storage efficiency, and stability (natural keys such as names or phone numbers can change). Patient IDs start at 12501, giving the impression of a mid-range record count in a new system.

11.3 Normalisation

The schema achieves Third Normal Form (3NF):

- 1NF: All columns hold atomic values. No repeating groups exist.
- 2NF: All non-key attributes are fully dependent on the entire primary key (each table has a single-column PK, so partial dependencies are impossible).
- 3NF: No transitive dependencies exist. For example, the doctor's speciality is stored as dept_id (a FK) rather than as a text string that would transitively depend on doctor_id via the department name.

11.4 ENUM for Appointment Status

Using ENUM('scheduled','in progress','postponed','completed') rather than a VARCHAR provides implicit domain constraint enforcement at the database level, reduces storage (ENUM is stored as a 1-2 byte integer internally), and produces self-documenting SQL. The four values cover the standard outpatient appointment lifecycle.

11.5 Overlap Prevention at Application Layer

The overlap check is implemented in the Express route rather than as a database trigger. This was likely chosen for simplicity and debuggability during academic development. A production system should additionally implement a BEFORE INSERT trigger (see Section 13) to ensure the constraint is enforced regardless of which application inserts the data.

11.6 Security Considerations

- Parameterised queries: All backend queries use the mysql2 placeholder syntax (?, [values]) instead of string concatenation, which prevents SQL injection attacks.
- CORS: CORS is enabled globally without origin restriction. In production, this should be locked to the frontend domain.
- Credentials: The database password is hardcoded in plain text. A production system would use environment variables (dotenv) or a secrets manager.
- Authentication: The current system has no user authentication layer. Adding JWT-based authentication would protect the API endpoints from unauthorised access.

11.7 Scalability Considerations

- Indexes on all FK columns ensure that JOIN operations scale logarithmically rather than linearly with data volume.
- Replacing the single db.createConnection with a connection pool (mysql.createPool) would allow the server to handle concurrent requests without connection exhaustion.
- The React frontend fetches entire tables on every operation (loadAll). For large datasets, pagination and server-side filtering would be required.

12. Views

12.1 Current State

No SQL VIEW objects are defined in the provided dump file. However, based on the query patterns in the backend API, the following views are proposed to improve maintainability and simplify recurring queries.

12.2 Proposed View 1: vw_appointment_detail

```
CREATE VIEW vw_appointment_detail AS
SELECT
    a.appointment_id,
    a.appointment_date,
    a.start_time,
    a.end_time,
    a.status,
    a.diagnosis,
    a.cost,
    p.name AS patient_name,
    p.phone AS patient_phone,
    d.name AS doctor_name,
    dep.name AS department_name,
    c.name AS clinic_name
FROM appointment a
LEFT JOIN patient p ON a.patient_id = p.patient_id
LEFT JOIN doctor d ON a.doctor_id = d.doctor_id
LEFT JOIN department dep ON d.dept_id = dep.dept_id
LEFT JOIN clinic c ON a.clinic_id = c.clinic_id;
```

Purpose: Replaces the multi-table JOIN currently written inline in the /appointments endpoint. Any query that needs a full appointment summary can SELECT from this view instead of repeating the JOIN logic.

Advantages: Reduces code duplication; allows the backend to issue simple SELECT * FROM vw_appointment_detail queries; if the underlying table structure changes, only the view definition needs updating.

12.3 Proposed View 2: vw_doctor_appointment_count

```
CREATE VIEW vw_doctor_appointment_count AS
SELECT
    d.doctor_id,
    d.name AS doctor_name,
    dep.name AS department,
    COUNT(a.appointment_id) AS total_appointments,
    SUM(a.cost) AS total_revenue,
    AVG(a.cost) AS avg_cost
FROM doctor d
LEFT JOIN appointment a ON d.doctor_id = a.doctor_id
```

```
LEFT JOIN department dep ON d.dept_id = dep.dept_id
GROUP BY d.doctor_id, d.name, dep.name;
```

Purpose: Provides a summary of each doctor's workload and revenue contribution. Useful for management reporting and future analytics endpoints.

12.4 Proposed View 3: vw_diagnosis_summary

```
CREATE VIEW vw_diagnosis_summary AS
SELECT
    diagnosis,
    COUNT(*) AS frequency,
    ROUND(AVG(cost), 2) AS avg_cost,
    MIN(appointment_date) AS first_seen,
    MAX(appointment_date) AS last_seen
FROM appointment
WHERE diagnosis IS NOT NULL
GROUP BY diagnosis
ORDER BY frequency DESC;
```

Purpose: Encapsulates the /analytics/diagnosis-trend query and extends it with average cost, earliest, and latest occurrence. The frontend could be enhanced to use this view for richer analytical displays.

13. Triggers

13.1 Current State

No triggers are defined in the provided SQL dump. The following section proposes three practical triggers with full MySQL implementation code and detailed explanations.

13.2 Proposed Trigger 1: Prevent Overlapping Appointments

```
DELIMITER $$

CREATE TRIGGER trg_no_overlap
BEFORE INSERT ON appointment
FOR EACH ROW
BEGIN
    DECLARE v_count INT;

    SELECT COUNT(*) INTO v_count
    FROM   appointment
    WHERE  doctor_id      = NEW.doctor_id
    AND    appointment_date = NEW.appointment_date
    AND    start_time      < NEW.end_time
    AND    end_time        > NEW.start_time;

    IF v_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Appointment overlaps an existing booking for this
doctor.';
    END IF;
END$$

DELIMITER ;
```

When it executes: BEFORE INSERT on the appointment table.

Purpose: Enforces the no-overlap business rule at the database level, independent of the application layer. This ensures data integrity even if direct SQL inserts or third-party tools bypass the Node.js API.

Mechanism: The trigger counts existing appointments for the same doctor, date, and overlapping time window using the standard interval intersection formula ($\text{start_time} < \text{new_end}$ AND $\text{end_time} > \text{new_start}$). If a conflict is found, SIGNAL SQLSTATE '45000' raises an application-level error, aborting the INSERT.

13.3 Proposed Trigger 2: Auto-Complete Past Appointments

```
DELIMITER $$

CREATE TRIGGER trg_auto_complete_status
BEFORE UPDATE ON appointment
```

```

FOR EACH ROW
BEGIN
    IF NEW.appointment_date < CURDATE()
        AND OLD.status IN ('scheduled', 'in progress')
    THEN
        SET NEW.status = 'completed';
    END IF;
END$$

DELIMITER ;

```

When it executes: BEFORE UPDATE on the appointment table.

Purpose: Automatically transitions appointments whose date has passed from 'scheduled' or 'in progress' to 'completed' when any column is updated. This prevents stale status values from accumulating over time.

13.4 Proposed Trigger 3: Audit Log on Appointment Deletion

```

-- First create an audit table
CREATE TABLE appointment_audit (
    audit_id          INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id    INT,
    deleted_at        DATETIME DEFAULT CURRENT_TIMESTAMP,
    deleted_by_app    VARCHAR(50) DEFAULT 'clinic_api'
);

DELIMITER $$

CREATE TRIGGER trg_appointment_audit
AFTER DELETE ON appointment
FOR EACH ROW
BEGIN
    INSERT INTO appointment_audit (appointment_id)
    VALUES (OLD.appointment_id);
END$$

DELIMITER ;

```

When it executes: AFTER DELETE on the appointment table.

Purpose: Records every deleted appointment in a separate audit table with a timestamp. This supports compliance requirements and allows administrators to investigate accidental deletions. The audit table is insert-only, providing a tamper-evident log.

14. System Workflow

14.1 End-to-End Process Description

The following workflow describes the complete lifecycle of a patient interaction within the Clinic Management System, from initial registration through to appointment completion and reporting.

Step 1 — System Initialisation

An administrator seeds the DEPARTMENT table with the 20 medical specialities. CLINIC records are then created and linked to their respective departments via dept_id.

Step 2 — Doctor Registration

Staff navigate to the Doctors tab on the dashboard and submit the Add Doctor form with name, specialisation, and phone. The frontend POSTs to /doctors; the backend inserts the record into the DOCTOR table with the corresponding dept_id.

Step 3 — Patient Registration

When a new patient arrives at the clinic, a receptionist navigates to the Patients tab. The Add Patient form captures name, phone, address, birth date, and occupation. The frontend POSTs to /patients; the backend inserts the record into the PATIENT table.

Step 4 — Appointment Booking

The receptionist selects the patient and doctor from dropdown lists (or enters their IDs), specifies the appointment date, start time, and end time. The frontend POSTs to /appointments. The backend first executes the overlap check query. If the slot is free, the appointment is inserted with status = 'scheduled'. If the slot is occupied, HTTP 400 is returned and the booking is rejected.

Step 5 — Appointment Execution

On the day of the appointment, the doctor consults the patient. The status is updated to 'in progress'. Following the consultation, the doctor records the diagnosis. The status is changed to 'completed' and the cost is finalised.

Step 6 — Analytics & Reporting

The dashboard automatically displays live counts of all entities. The /analytics/diagnosis-trend endpoint returns the frequency of each diagnosis, ranked by occurrence — this identifies the most common health conditions seen at the clinic. The /analytics/revenue endpoint sums all appointment costs to produce total revenue figures for financial reporting.

Step 7 — Record Maintenance

Administrators can delete patient or doctor records from their respective tabs. Referential integrity constraints (RESTRICT) prevent deletion of patients or doctors who have existing appointments, protecting historical medical records.

14.2 Data Flow Diagram (Textual)

Browser (React) → HTTP Request → Express API → mysql2 → MySQL (InnoDB) → Results → Express API → JSON Response → React State → DOM Update

15. Advanced SQL Questions

Question 1 — Preventing Overlapping Doctor Appointments

Problem: How would you prevent two appointments for the same doctor from overlapping on the same date?

15.1.1 Application-Layer Solution (already implemented)

```
-- Node.js backend executes this before every INSERT:
SELECT * FROM Appointment
WHERE  doctor_id      = ?           -- same doctor
AND    appointment_date = ?       -- same date
AND (   start_time    < ?           -- existing starts before new ends
      AND end_time    > ? ) ;      -- existing ends after new starts
-- If result.length > 0 → reject with HTTP 400
```

15.1.2 Database-Level Trigger Solution

```
DELIMITER $$
CREATE TRIGGER trg_no_overlap
BEFORE INSERT ON appointment
FOR EACH ROW
BEGIN
    DECLARE v_count INT DEFAULT 0;
    SELECT COUNT(*) INTO v_count
    FROM    appointment
    WHERE   doctor_id      = NEW.doctor_id
    AND     appointment_date = NEW.appointment_date
    AND     start_time     < NEW.end_time
    AND     end_time       > NEW.start_time;
    IF v_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Time slot already booked for this doctor.';
    END IF;
END$$
DELIMITER ;
```

Explanation: The BEFORE INSERT trigger fires before every new row is written to the appointment table. The overlap check uses the standard interval intersection formula: two intervals [S1,E1] and [S2,E2] overlap when S1 < E2 AND S2 < E1. Using SIGNAL SQLSTATE '45000' raises a user-defined exception, which rolls back the INSERT and returns an error to the application. This database-level enforcement is independent of the application, making it robust against direct SQL access.

Question 2 — Patient Information with Appointments

Problem: Write a query to fetch each patient's information along with their appointment history.

```

SELECT
    p.patient_id,
    p.name           AS patient_name,
    p.phone,
    p.birth_date,
    p.job,
    a.appointment_id,
    a.appointment_date,
    a.start_time,
    a.end_time,
    a.status,
    a.diagnosis,
    a.cost,
    d.name           AS doctor_name,
    dep.name         AS department
FROM    patient     p
LEFT JOIN appointment a  ON p.patient_id = a.patient_id
LEFT JOIN doctor      d  ON a.doctor_id  = d.doctor_id
LEFT JOIN department  dep ON d.dept_id   = dep.dept_id
ORDER BY p.patient_id, a.appointment_date DESC;

```

JOIN Explanation:

LEFT JOIN is used so that patients who have no appointments still appear in the result set (with NULL values for appointment columns). An INNER JOIN would exclude patients with no appointment history.

- p JOIN a: links each patient to their appointments via patient_id.
- a JOIN d: resolves the doctor's name from the doctor table via doctor_id.
- d JOIN dep: resolves the department name via the doctor's dept_id.

Expected Output: One row per appointment per patient, with patient demographics repeated across rows. Patients with no appointments appear once with NULL appointment columns.

Question 3 — View: Upcoming Appointment Count per Doctor

```

CREATE OR REPLACE VIEW vw_upcoming_appointments AS
SELECT
    d.doctor_id,
    d.name           AS doctor_name,
    dep.name         AS department,
    COUNT(a.appointment_id) AS upcoming_count
FROM    doctor      d
LEFT JOIN appointment a
    ON  a.doctor_id = d.doctor_id
    AND a.appointment_date BETWEEN CURDATE()
                                AND DATE_ADD(CURDATE(), INTERVAL 1 MONTH)
    AND a.status      IN ('scheduled', 'in progress')
LEFT JOIN department dep ON d.dept_id = dep.dept_id
GROUP BY d.doctor_id, d.name, dep.name

```

```
ORDER BY upcoming_count DESC;
```

GROUP BY Explanation:

GROUP BY d.doctor_id, d.name, dep.name collapses all appointment rows for each doctor into a single summary row. COUNT(a.appointment_id) counts only non-NULL appointment_id values (i.e., actual appointments), so doctors with no upcoming appointments return a count of 0 rather than being excluded.

Aggregation: COUNT is the aggregate function. It operates per group defined by the GROUP BY clause. The BETWEEN filter restricts counting to the next 30 days. The status filter excludes already-completed or postponed appointments.

Expected Output: One row per doctor showing how many upcoming appointments they have in the next month, ordered from busiest to least busy — useful for workload management.

16. Conclusion

The Clinic Management System presented in this report demonstrates a well-structured, end-to-end web application for managing the core entities of a healthcare facility. The database design achieves Third Normal Form, ensures referential integrity through InnoDB foreign key constraints, and models the real-world relationships between patients, doctors, departments, clinics, and appointments accurately.

16.1 Database Design Quality

The relational schema is clean, normalised, and appropriately constrained. The use of surrogate integer primary keys, ENUM types for status fields, and DECIMAL for monetary values reflects sound database design practice. The appointment table's three foreign keys correctly implement the associative entity that lies at the heart of the clinic's operations.

16.2 Backend Implementation

The Express.js API provides a complete set of CRUD operations for all major entities. The parameterised query approach prevents SQL injection, and the application-layer overlap check adds meaningful business logic beyond simple persistence. The code is readable and well-organised by entity.

16.3 Frontend Implementation

The React dashboard offers a functional, visually polished interface with a modern dark theme. The tab-based navigation, real-time data loading via fetch, and immediate refresh after mutations provide a responsive user experience. The dashboard summary cards provide at-a-glance system status.

16.4 Strengths

- Clean separation between the four layers: database, API, frontend, and presentation.
- Referential integrity enforced at the database level, protecting data consistency.
- Analytics endpoints (diagnosis trends, revenue) add immediate business value beyond basic CRUD.
- Modern, responsive UI design that would be suitable for a clinical reception environment.

16.5 Potential Future Improvements

- Implement database-level triggers for overlap prevention and audit logging (as proposed in Section 13).
- Add SQL VIEWS (proposed in Section 12) to centralise complex JOIN logic.
- Introduce JWT-based authentication to secure API endpoints.
- Migrate from single connection to a connection pool for production scalability.
- Add server-side pagination and filtering for large datasets.

- Implement UPDATE endpoints for editing existing records.
- Add input validation and error feedback in the frontend forms.
- Store database credentials in environment variables (.env) rather than source code.
- Introduce UNIQUE constraints on doctor phone numbers and patient national IDs to prevent duplicate registrations.

17. External Libraries and Sources

This project was developed by the authors listed above. All database design, SQL scripts, backend APIs, frontend implementation, and project integration were completed as part of the course requirements.

The following external technologies and libraries were used:

Database

- MySQL

Backend

- Node.js
- Express.js
- MySQL2
- CORS

Frontend

- React.js

Development Tools

- MySQL Workbench
- Visual Studio Code
- Git & GitHub

References

Official documentation was consulted during development:

- MySQL Documentation: <https://dev.mysql.com/doc/>
- Node.js Documentation: <https://nodejs.org/docs/>
- Express.js Documentation: <https://expressjs.com/>
- React Documentation: <https://react.dev/>
- MySQL2 Documentation: <https://github.com/sidorares/node-mysql2>

All code submitted in this repository represents the authors' own work, except for the use of the above libraries, frameworks, and officially documented examples used for learning and implementation purposes.

— End of Report —